Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт информационных технологий и кибербезопасности
Кафедра инфокоммуникаций

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1
дисциплины «Объектно-ориентированное программирование»
Вариант 7

Выполнил:
Хубиев Роберт Эльбрусович
3 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и вычислительная техника»,
направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____
(подпись)

Руководитель практики:
Воронкин Р.А., доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии

_____
(подпись)

Отчет защищен с оценкой _____ Дата защиты_____

Ставрополь, 2026 г.

Тема: элементы объектно-ориентированного программирования в языке Python

Ссылка на репозиторий: https://github.com/truebobsuncle/oop1.git

Цель: приобретение навыков по работе с классами и объектами при написании программ с помощью языка программирования Python версии 3.x.

Порядок выполнения работы:

Примеры

Класс Rational для работы с рациональными дробями. Реализует операции арифметики (сложение, вычитание, умножение, деление) и операции сравнения (больше, меньше, равно)

```
class Rational:
    def __init__(self, a=0, b=1):
        a = int(a)
        b = int(b)

        if b == 0:
            raise ValueError()

        self.__numerator = abs(a)
        self.__denominator = abs(b)

        self.__reduce()

    # Сокращение дроби
    def __reduce(self):
        # Функция для нахождения наибольшего общего делителя
        def gcd(a, b):
            if a == 0:
                return b
            elif b == 0:
                return a
            elif a >= b:
                return gcd(a % b, b)
            else:
                return gcd(a, b % a)

        c = gcd(self.__numerator, self.__denominator)

        self.__numerator //= c
        self.__denominator //= c

    @property
    def numerator(self):
        return self.__numerator

    @property
    def denominator(self):
        return self.__denominator

    # Прочитать значение дроби с клавиатуры. Дробь вводится как a/b.
```

```
    def read(self, prompt=None):
        line = input() if prompt is None else input(prompt)
        parts = list(map(int, line.split("/", maxsplit=1)))

        if parts[1] == 0:
            raise ValueError()

        self.__numerator = abs(parts[0])
        self.__denominator = abs(parts[1])

        self.__reduce()

    # Вывести дробь на экран
    def display(self):
        print(f"{self.__numerator}/{self.__denominator}")

    # Сложение обыкновенных дробей.
    def add(self, rhs):
        if isinstance(rhs, Rational):
            a = self.numerator * rhs.denominator + self.denominator *
rhs.numerator
            b = self.denominator * rhs.denominator
            return Rational(a, b)
        else:
            raise ValueError()

    # Вычитание обыкновенных дробей.
    def sub(self, rhs):
        if isinstance(rhs, Rational):
            a = self.numerator * rhs.denominator - self.denominator *
rhs.numerator
            b = self.denominator * rhs.denominator

            return Rational(a, b)
        else:
            raise ValueError()

    # Умножение обыкновенных дробей.
    def mul(self, rhs):
        if isinstance(rhs, Rational):
            a = self.numerator * rhs.numerator
            b = self.denominator * rhs.denominator

            return Rational(a, b)
        else:
            raise ValueError()

    # Деление обыкновенных дробей.
    def div(self, rhs):
        if isinstance(rhs, Rational):
            a = self.numerator * rhs.denominator
            b = self.denominator * rhs.numerator

            return Rational(a, b)
        else:
            raise ValueError()

    # Отношение обыкновенных дробей.
    def equals(self, rhs):
        if isinstance(rhs, Rational):
            return (self.numerator == rhs.numerator) and (self.denominator ==
rhs.denominator)
        else:
            return False
```

```
    def greater(self, rhs):
        if isinstance(rhs, Rational):
            v1 = self.numerator / self.denominator
            v2 = rhs.numerator / rhs.denominator

            return v1 > v2
        else:
            return False

    def less(self, rhs):
        if isinstance(rhs, Rational):
            v1 = self.numerator / self.denominator
            v2 = rhs.numerator / rhs.denominator
            return v1 < v2
        else:
            return False


if __name__ == "__main__":
    r1 = Rational()
    r1.read('Введите первую дробь: ')

    r2 = Rational()
    r2.read('Введите вторую дробь: ')

    print('Сумма: ')
    r3 = r2.add(r1)
    r3.display()

    print('Разность: ')
    r4 = r2.sub(r1)
    r4.display()

    print('Произведение: ')
    r5 = r2.mul(r1)
    r5.display()

    print('Частное: ')
    r6 = r2.div(r1)
    r6.display()
```

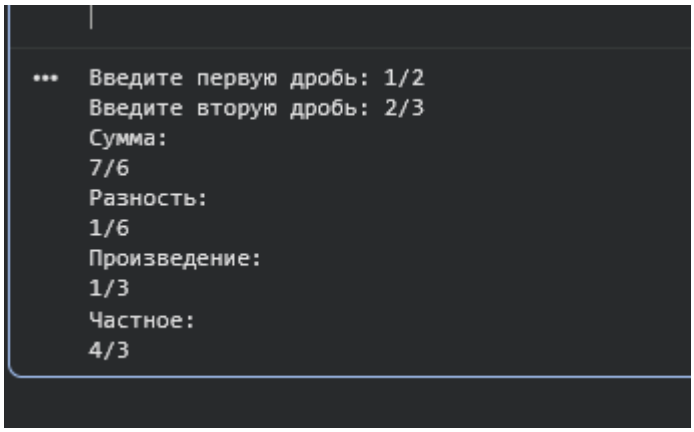


Рисунок 1. Результат выполнения примера

Индивидуальные задания

Задание 1: реализовать класс с двумя полями, с именами first и second. Требуется реализовать тип данных с помощью такого класса. Реализовать методы:

1) метод инициализации __init__; метод должен значения аргументов на корректность;

2) ввод с клавиатуры read;

3) вывод на экран display.

Реализовать внешнюю функцию с именем make_pair(). Функция должна получать в качестве аргументов значения для полей структуры и возвращать структуру требуемого типа. При передаче ошибочных параметров следует выводить сообщение и заканчивать работу.

Поле first — дробное число, левая граница диапазона; поле second — дробное число, правая граница диапазона. Реализовать метод rangecheck() — проверку заданного числа на принадлежность диапазону.

Решение:

```
class Pair:
    def __init__(self, first, second):
        if not isinstance(first, (int, float)) or not isinstance(second,
(int, float)):
            raise ValueError("Поля должны быть числами.")
        if first > second:
            raise ValueError("Левая граница не может быть больше правой.")
        self.first = first
        self.second = second

    def read(self):
        f = float(input("first: "))
        s = float(input("second: "))
        self.__init__(f, s)

    def display(self):
        print(f"first = {self.first}, second = {self.second}")

    def rangecheck(self, number):
        return self.first <= number <= self.second


def make_pair(cls, *args):
```

```
    try:
        return cls(*args)
    except Exception as e:
        print(f"Ошибка: {e}")
        return None


if __name__ == '__main__':
    p = make_pair(Pair, 5.0, 15.0)
    if p:
        p.display()
        print(p.rangecheck(10))
        print(p.rangecheck(20))

    p2 = make_pair(Pair, 0, 0)
    if p2:
        p2.read()
        p2.display()

    make_pair(Pair, 100, 50)
```

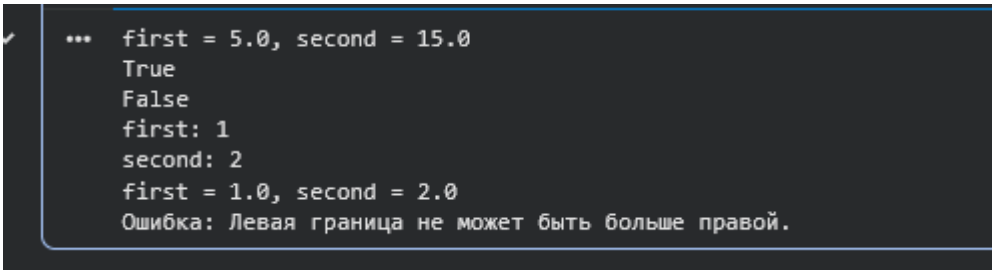


Рисунок 2. Результат выполнения тестового кода

Задание 2. Создать класс Date для работы с датами в формате «год.месяц.день». Дата представляется структурой с тремя полями типа unsigned int: для года, месяца и дня. Класс должен включать не менее трех функций инициализации: числами, строкой вида «год.месяц.день» (например, «2004.08.31») и датой. Обязательными операциями являются: вычисление даты через заданное количество дней, вычитание заданного количества дней из даты, определение високосности года, присвоение и получение отдельных частей (год, месяц, день), сравнение дат (равно, до, после), вычисление количества дней между датами

Решение:

```
class Date:
```

```
    def __init__(self, *args):
        """Метод инициализации. Поддерживает создание из 3 чисел или из
строки."""
        if len(args) == 3:

            self._date = datetime(args[0], args[1], args[2])
        elif len(args) == 1 and isinstance(args[0], str):

            self._date = datetime.strptime(args[0], "%Y.%m.%d")
        else:
            raise ValueError("Ожидается 3 числа (год, месяц, день) или
строка формата 'YYYY.MM.DD'")


    def read(self):
        """Ввод с клавиатуры"""
        date_str = input("Введите дату в формате 'год.месяц.день'
(например, 2004.08.31): ")
        self._date = datetime.strptime(date_str, "%Y.%m.%d")

    def display(self):
        """Вывод на экран в заданном формате"""
        print(self._date.strftime("%Y.%m.%d"))


    def add_days(self, days):
        """Вычисление даты через заданное количество дней"""
        return Date((self._date + timedelta(days=days)).year,
                    (self._date + timedelta(days=days)).month,
                    (self._date + timedelta(days=days)).day)

    def subtract_days(self, days):
        """Вычитание заданного количества дней из даты"""
        return self.add_days(-days)

    def is_leap_year(self):
        """Определение високосности года"""
        year = self._date.year
        return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)

    def get_year(self): return self._date.year
    def get_month(self): return self._date.month
    def get_day(self): return self._date.day

    def set_year(self, year): self._date = self._date.replace(year=year)
    def set_month(self, month): self._date =
self._date.replace(month=month)
    def set_day(self, day): self._date = self._date.replace(day=day)

    def __eq__(self, other): return self._date == other._date
    def __lt__(self, other): return self._date < other._date
```

```
    def __gt__(self, other): return self._date > other._date

    def days_between(self, other):
        """Вычисление количества дней между датами"""
        return abs((self._date - other._date).days)


def make_date(*args):
    """Внешняя функция для создания объекта с проверкой ошибок"""
    try:
        return Date(*args)
    except Exception as e:
        print(f"Ошибка: {e}")
        return None



if __name__ == '__main__':
    print("--- Демонстрация работы класса Date ---")


    d1 = make_date(2004, 8, 31)
    d2 = make_date("2023.01.01")
    print("Дата 1:", end=" "); d1.display()
    print("Дата 2:", end=" "); d2.display()


    print(f"\nГод {d1.get_year()} високосный? -> {d1.is_leap_year()}")
    print(f"Год {d2.get_year()} високосный? -> {d2.is_leap_year()}")


    print("\nПрибавляем 10 дней к Дате 1:")
    d1.add_days(10).display()

    print("Вычитаем 1 день из Даты 2:")
    d2.subtract_days(1).display()


    print(f"\nДней между 2004.08.31 и 2023.01.01: {d1.days_between(d2)}")


    print(f"Дата 1 > Дата 2? -> {d1 > d2}")
    print(f"Дата 1 == Дата 2? -> {d1 == d2}")


    print("\nМеняем год у Даты 2 на 2024:")
    d2.set_year(2024)
    d2.display()


    print("\nПроверка ввода с клавиатуры (read):")
```

```
    d3 = make_date(1, 1, 1) # создаем заглушку
    d3.read()
    print("Вы ввели:", end=" "); d3.display()
```

```
Дата 1: 2004.08.31
Дата 2: 2023.01.01

Год 2004 високосный? -> True
Год 2023 високосный? -> False

Прибавляем 10 дней к Дате 1:
2004.09.10
Вычитаем 1 день из Даты 2:
2022.12.31

Дней между 2004.08.31 и 2023.01.01: 6697
Дата 1 > Дата 2? -> False
Дата 1 == Дата 2? -> False

Меняем год у Даты 2 на 2024:
2024.01.01

Проверка ввода с клавиатуры (read):
Введите дату в формате 'год.месяц.день' (например, 2004.08.31): 2001.11.11
Вы ввели: 2001.11.11
```

Рисунок 3. Результата выполнения кода

Ответы на контрольные вопросы:

1. Как осуществляется объявление класса в языке Python?

Для создания класса применяется ключевое слово class, после которого указывается имя класса с заглавной буквы (по правилам CamelCase), двоеточие и блок кода с отступами:

```
class MyClass:
    # Атрибуты (переменные) класса
    variable = 10

    # Методы класса
    def do_something(self):
        pass
```

2. Чем атрибуты класса отличаются от атрибутов экземпляра?

Атрибуты класса создаются непосредственно внутри тела класса (вне методов). Они являются общими для всех объектов этого класса и разделяют одно значение между ними (работают как глобальные константы или значения по умолчанию).

Атрибуты экземпляра объявляются внутри методов (обычно в __init__) через ссылку self. Они уникальны для каждого конкретного объекта и хранят его индивидуальное состояние (например, координаты конкретного окна).

3. Каково назначение методов класса?

Методы класса представляют собой функции, которые описывают поведение объектов. Они используются для взаимодействия с данными объекта: могут считывать его текущие свойства, изменять состояние полей (например, перемещать координаты или менять цвет) или выполнять связанные с этим объектом вычисления.

4. Для чего предназначен метод __init__() класса?

Метод __init__ выполняет роль конструктора класса. Он вызывается автоматически в момент создания нового объекта. Его главная задача — инициализировать (задать) начальные значения для атрибутов экземпляра и выполнить базовые проверки данных при старте.

5. Каково назначение self?

Параметр self — это ссылка на текущий конкретный объект (экземпляр), для которого вызывается метод. Через self программа понимает, к атрибутам и методам какого именно объекта нужно обратиться изнутри кода класса. Он всегда идет первым аргументом в методах экземпляра.

6. Как добавить атрибуты в класс?

Чтобы добавить атрибут класса, его достаточно объявить прямой записью внутри тела класса.

Чтобы добавить атрибут экземпляра, его создают внутри методов (чаще всего в конструкторе __init__) путем присвоения значения через ссылку self, например: self.name = value.

7. Как осуществляется управление доступом к методам и атрибутам в языке Python?

В Python нет строгого разделения на приватные и публичные данные на уровне компилятора, доступ регулируется соглашением об именовании:

Публичные: пишутся без подчеркиваний (например, title, shift).

Защищенные: начинаются с одного подчеркивания (например, _color).

Приватные (скрытые на уровне класса): начинаются с двух подчеркиваний (например, __status). Python автоматически подменяет их имена, чтобы к ним нельзя было обратиться напрямую извне.

8. Каково назначение функции isinstance?

Встроенная функция isinstance(object, classinfo) используется для проверки типа данных. Она возвращает True, если проверяемый объект относится к указанному классу или к любому из его подклассов (наследников). Это позволяет безопасно проверять типы аргументов (например, перед выполнением арифметических действий или валидацией ввода).

Вывод: в процессе выполнения лабораторной работы были успешно усвоены фундаментальные принципы объектно-ориентированного программирования и получены практические навыки разработки программ на Python 3.x.